

คำสั่ง Git ที่ต้องรู้ (หลักพาเรโต 80/20)

รวมคำสั่ง Git 20% ที่ถูกใช้งานจริงประมาณ 80% ของงานประจำวัน — สำหรับรันใน Terminal ของ VS Code

หลักพาเรโต (Pareto Principle) กับ Git: ในการทำงานจริง คุณจะใช้คำสั่งเพียงไม่กี่ตัวซ้ำ ๆ ทุกวัน (status, add, commit, push, pull) ส่วนคำสั่งที่เหลือ (rebase, cherry-pick, bisect ฯลฯ) ใช้เฉพาะกรณีพิเศษเท่านั้น เอกสารนี้จึงคัดเฉพาะคำสั่งกลุ่มแรกที่คุณค่าต่อการจำมากที่สุด จัดกลุ่มตามลำดับการทำงานจริงในแต่ละวัน

1 ตรวจสอบสถานะงาน (ใช้บ่อยที่สุด — เช็คก่อนทำอะไรทุกครั้ง)

เปิด Terminal ใน VS Code ด้วย **Ctrl + `** (Windows) หรือ **Control + `** (Mac) แล้วอยู่ในโฟลเดอร์โปรเจกต์เสมอ

คำสั่ง	คำอธิบาย	ตัวอย่างการใช้
<code>git status</code>	ดูว่ามีไฟล์ไหนถูกแก้ไข/เพิ่มใหม่ และยังไม่ถูก commit บ้าง ควรรันก่อนและหลังทำงานทุกครั้ง	รันเปล่า ๆ ไม่ต้องใส่ argument
<code>git log --oneline</code>	ดูประวัติการ commit แบบย่อ บรรทัดละ 1 commit เพื่อดูว่าใครทำอะไรไปแล้วบ้าง	<code>git log --oneline -10</code> (ดู 10 รายการล่าสุด)
<code>git diff</code>	ดูรายละเอียดว่าไฟล์ที่แก้ไข เปลี่ยนบรรทัดไหนไปบ้าง ก่อนตัดสินใจ commit	<code>git diff --stat</code> (ดูสรุปจำนวนบรรทัดที่เปลี่ยน)

2 บันทึกการเปลี่ยนแปลง (Commit)

ทำทุกครั้งหลังแก้ไขไฟล์เสร็จเป็นชิ้น ๆ ไม่ต้องรอทำเยอะแล้วค่อย commit รวด

คำสั่ง	คำอธิบาย	ตัวอย่างการใช้
<code>git add <file></code>	นำไฟล์ที่แก้ไขเข้า "staging area" เตรียมพร้อมสำหรับ commit ควรระบุชื่อไฟล์ตรง ๆ ดีกว่าใช้ <code>-A</code> เพื่อไม่ให้หลุดไฟล์ที่ไม่ตั้งใจ	<code>git add docs/backlog.md</code>
<code>git commit -m "ข้อความ"</code>	บันทึกการเปลี่ยนแปลงที่ staged ไว้ลงประวัติ พร้อมข้อความอธิบายว่าทำไมถึงแก้ (ไม่ใช่แค่แก้อะไร)	<code>git commit -m "เพิ่ม PDPA"</code>
<code>git commit --amend</code>	แก้ไข commit ล่าสุดที่เพิ่งทำไป (เช่น พิมพ์ข้อความผิด) ใช้เฉพาะ commit ที่ยังไม่ push เท่านั้น	ใช้เมื่อยังไม่ได้ push ออกไป

3 ชิงค์กับ GitHub (Remote)

โปรเจกต์นี้เชื่อมกับ `github.com/aitomprompts-cmyk/my-coffee-store2t` ผ่าน remote ชื่อ `origin` เรียบร้อยแล้ว

คำสั่ง	คำอธิบาย	ตัวอย่างการใช้
<code>git pull</code>	ดึงงานล่าสุดจาก GitHub มารวมกับเครื่องเรา ควรทำก่อนเริ่มงานทุกครั้งที่เปิดเครื่องใหม่	<code>git pull origin main</code>
<code>git push</code>	ส่ง commit ที่ทำในเครื่องเราขึ้นไปเก็บบน GitHub ให้คนอื่นเห็นและมีไฟล์สำรอง	<code>git push origin main</code>
<code>git remote -v</code>	ตรวจสอบว่าโปรเจกต์เชื่อมกับ GitHub ตัวไหนอยู่ (fetch/push URL)	รันเปล่า ๆ ตรวจสอบก่อน push ครั้งแรก
<code>git clone <url></code>	คัดลอกโปรเจกต์จาก GitHub ลงมาที่เครื่องใหม่ (ใช้ครั้งแรกครั้งเดียวต่อเครื่อง)	<code>git clone https://github.com/...</code>

4 การแตกกิ่งงาน (Branch)

ใช้เมื่อจะลองทำฟีเจอร์ใหม่โดยไม่กระทบโค้ด/เอกสารหลักใน main

คำสั่ง	คำอธิบาย	ตัวอย่างการใช้
<code>git branch</code>	ดูรายชื่อ branch ทั้งหมดในเครื่อง และดูว่าตอนนี้อยู่ branch ไหน	รันเปล่า ๆ
<code>git switch -c <name></code>	สร้าง branch ใหม่และสลับไปทำงานที่ branch นั้นทันที (คำสั่งใหม่แทน <code>checkout -b</code>)	<code>git switch -c feature/pdpa</code>
<code>git switch main</code>	สลับกลับมาที่ branch หลัก	<code>git switch main</code>
<code>git merge <branch></code>	รวมงานจาก branch อื่นเข้ามาที่ branch ปัจจุบัน (ทำตอนที่ branch ปลายทาง)	<code>git merge feature/pdpa</code>

5 แก้ไข/กู้คืนเมื่อทำพลาด

กลุ่มนี้ใช้ไม่บ่อย แต่สำคัญมากตอนพลาด ควรอ่านคำอธิบายให้ครบก่อนใช้จริง

คำสั่ง	คำอธิบาย	ตัวอย่างการใช้
<code>git restore <file></code>	ยกเลิกการแก้ไขไฟล์ที่ยังไม่ได้ add กลับไปเหมือนเดิมตามครั้งล่าสุดที่ commit	<code>git restore backlog.md</code>
<code>git restore --staged <file></code>	เอาไฟล์ออกจาก staging area (ยกเลิก git add) แต่ยังเก็บการแก้ไขในไฟล์ไว้	<code>git restore --staged backlog.md</code>
<code>git stash</code>	เก็บงานที่แก้ค้างไว้ชั่วคราวโดยไม่ต้อง commit เพื่อต้องสลับ branch อื่น ๆ	<code>git stash</code> แล้ว <code>git stash pop</code> เพื่อเอากลับมา
<code>git reset --soft HEAD~1</code>	ยกเลิก commit ล่าสุด 1 อัน แต่ยังเก็บการแก้ไขไฟล์ไว้ใน staging area ให้แก้ commit message หรือรวม commit ใหม่ได้	ใช้เมื่อ commit ล่าสุดยังไม่ push

ข้อควรระวัง: คำสั่งที่มี `--hard`, `push --force`, หรือ `clean -f` จะลบงานทั้งแบบกู้คืนยาก ไม่ควรใช้จนกว่าจะแน่ใจ 100% และควร `git status` เช็คก่อนทุกครั้ง

เคล็ดลับ VS Code: นอกจากพิมพ์คำสั่งเอง ยังใช้แถบ *Source Control* (ไอคอนรูปกิ่งไม้ด้านซ้าย หรือกด **Ctrl+Shift+G**) เพื่อ stage, commit, push ผ่าน GUI ได้เช่นกัน โดยเบื้องหลังคือคำสั่งชุดเดียวกับในตารางนี้ทั้งหมด